HW06 - AI Audit Report (Mandatory Appendix)

Student ID: 23127184 · Assignment: HW06 - API Testing · AI policy: Open

Declaration

> I use AI tools for the following tasks.

Non-AI work includes the real Newman executions and the student-only evidence listed in the main report.

Tools declared

Tool	Model / version	Used for
Claude Code	Opus 5	Repository scaffolding, harness scripts, collection skeletons, source reading
		Test-case generation for API 1
		Test-case generation for API 2
		Test-case generation for API 3
		Test-case generation for API 4
Codex	GPT-5	Case-by-case audit, student-designed extensions, execution triage, phase/report completion

Non-AI tools used: Postman, Newman + newman-reporter-htmlextra, Node.js, Python, Git, GitHub Actions.

Interaction log

Every interaction needs: tool name, date and time, the prompt verbatim, and the AI output. Keep them in order. Long outputs go in `docs/ai-audit/transcripts/` with a link from the table row.

AI-001 | Scaffolding the HW06 workspace

• Tool: Claude Code (Opus 5)

• Date/time: 2026-08-23

• Prompt: "đọc yêu cầu hw06 và setup cho mình" (read the HW06 requirements

and set up the workspace), followed by the API selection FR-01, FR-06, FR-11, FR-13 and the decision to use Postman cloud features.

• Output: The HW06 directory tree, `package.json` + Newman toolchain, four

Postman collection skeletons with the `X-Student-Id` pre-request harness, Postman environments, data-driven CSV fixtures, the CI workflow and green-gate manifest, the phase/audit/critique/CI document skeletons, and a bug report seeded from a verified smoke run.

• My review: Checked the selected endpoints against the supplied API and

FR/SEC sources, verified that each collection inherits the mandatory header harness, installed the local runner, and executed the initial smoke suite.

AI-002 | Generating the test cases for API 1 (FR-01, `POST /api/register`)

• Tool: Claude Code (Opus 5)

• Date/time: 2026-08-23

• Prompt: "bắt đầu generate test cases cho API 1, càng nhiều càng tốt, x2

yêu cầu đề bài" (start generating test cases for API 1, as many as possible, 2x the assignment's requirement).

• Step decomposition: the AI was not given that instruction as a single

prompt to act on. It was driven through the five stages defined in the `api-test-generator` skill - contract restatement, domain partitions per parameter, state transitions, security per SEC id, schema validation - with each stage's output feeding the next. The per-stage goals and outputs are recorded in `docs/phases/api1-fr01-register/01-generate.md` §2.

• Standing constraint imposed on the AI: every expected result must be

derived from `api_specification.md` and the FR-01 / SEC-01..07 rules, never from the SUT's observed responses.

• Output: 121 test cases (3.5x the brief's minimum of 35) - 79 domain

partition, 10 state transition, 20 security, 12 schema - expressed as a case specification in `scripts/cases/api1_fr01_register.py` and rendered into the Postman collection, the Excel sheet and a coverage tally. 11 cases carry an explicit specification-gap flag instead of an invented expectation.

• Verification performed before accepting the output: all 121 cases were

executed against the seeded SUT. 68 passed, 53 failed, and every failure was inspected to confirm it came from the SUT rather than from a broken fixture. Two defects not previously known were found this way (HTTP 500 on a non-JSON `Content-Type`; an HTML stack-trace page on malformed JSON).

• Issue already identified in the AI's output: `A1-SEC-013` fails for a

different reason than its title claims - see `01-generate.md` §7 item 1.

• My review: Completed in the final case-by-case register. The audit marked

ambiguous FR-01 limits INCOMPLETE, corrected the cause/title of `A1-SEC-013`, and kept failed assertions only when their oracle remained traceable to FR-01 or a SEC rule.

AI-003 | Audit, extension, re-execution, and report completion

• Tool: Codex (GPT-5)

• Date/time: 2026-08-23, ICT (Asia/Saigon)

• Prompt: "Audit từng case theo VALID / INVALID / INCOMPLETE. Thêm tối

thiểu 5 test case do sinh viên tự thiết kế cho mỗi API. Điền kết quả thực thi và bug vào các phase document. Hoàn thiện báo cáo chính; cập nhật README; làm evidence thật, sơ đồ generator, video demo và commit riêng cho từng phase; hãy làm các việc này cho mình một cách chỉn chu để được full điểm."

• Output: An audit decision and reason for all 386 final cases; correction

of duplicate `A2-DP-006` and misattributed `A1-SEC-013`; 20 independently marked student-designed cases; regenerated Postman collections and coverage reports; a verified full run (1,674/1,802 assertions passed) and green gate (1,271/1,271); phase 2-4 documents, bug clustering, README, main report, and submission/evidence instructions.

• My review: Verified the generated collections compile, ran both suites

against a freshly seeded local backend, retained the raw Newman JSON/HTML/log artefacts, and did not fabricate cloud screenshots, CI run URLs, a self-drawn diagram, or a narrated video.

Review discipline applied

For each batch of AI output, record which of these was done:

- [x] Read every generated test case against `refs/spec/api_specification.md`
- [x] Checked expected results against the requirement document, not against

the SUT's observed behaviour

- [x] Executed the cases rather than trusting the AI's predicted results
- [x] Corrected or discarded the cases that did not survive review
- [x] Recorded the corrections in the phase-2 audit documents

Bloom-AI level evidence

Level	Where it is evidenced
G9.2 Apply	Driving the AI step by step through partitions, state transitions, security and schema — `docs/phases/*/01-generate.md`
G9.3 Analyse	VALID / INVALID / INCOMPLETE audit with reasoning — `docs/phases/*/02-audit.md`
G9.4 Collaborate	Extension cases and the diagnosis of why the AI missed them — `docs/phases/*/03-extend.md`
G9.5 Create	The AI-driven test generator design — `docs/design/GENERATOR_DESIGN.md` and `.claude/skills/`